

Angular

Fundamentos del framework

Basado en la documentación oficial · angular.dev

¿Qué es Angular?

Framework de desarrollo frontend mantenido por Google. Escrito en TypeScript. Diseñado para construir Single Page Applications (SPA) robustas y escalables.

Componentes

Unidad básica de construcción. Cada componente encapsula HTML, CSS y lógica. Jerarquía en forma de árbol.

TypeScript

Lenguaje principal. Proporciona tipado estático, decoradores, interfaces y mejores herramientas de desarrollo.

SPA

Una sola página HTML. Angular maneja el enrutamiento del lado del cliente sin recargar el navegador.

DI

Sistema de Inyección de Dependencias integrado. Servicios, pipes y otros elementos se registran y se inyectan automáticamente.

Reactividad

Dos sistemas: RxJS (Observables) para flujos asíncronicos y Signals para estado reactivo simple.

Standalone

Desde Angular 14+ los componentes pueden ser independientes, sin necesidad de NgModules. Angular 17+ promueve standalone como default.

Arquitectura general

Una aplicación Angular se organiza en torno a estos bloques fundamentales:

Componentes

Controlan una parte de la pantalla (vista).
Cada componente define su propio template, estilos y lógica.

Templates

HTML extendido con directivas, binding, pipes y control de flujo (`@if` , `@for`).

Servicios

Clases con lógica reutilizable (HTTP, estado, utilidades). Se inyectan en componentes y otros servicios.

Routing

Navegación entre vistas. Soporte para rutas lazy, guards, parámetros y resolución de datos.

Pipes

Transforman valores en el template: `date` , `currency` , `json` , o pipes personalizados.

Directivas

Atributos que modifican el DOM: `ngClass` , `ngStyle` , o directivas estructurales nativas.

En versiones recientes (Angular 17+), las directivas estructurales nativas `*ngIf` / `*ngFor` se reemplazan por `@if` / `@for` en el template.

Componentes

Un componente es una clase TypeScript decorada con `@Component` . Controla un fragmento de la interfaz.

Decorador `@Component`

```
@Component({
  selector: 'app-user-card',
  standalone: true,
  imports: [CommonModule],
  templateUrl: './user-card.html',
  styleUrls: ['./user-card.css']
})
export class UserCardComponent {
  @Input() name: string = '';
  @Output() click = new EventEmitter<void>();
}
```

Propiedad

<code>selector</code>	Etiqueta HTML para usar el componente
<code>standalone</code>	Indica si es independiente (sin NgModule)
<code>imports</code>	Módulos/componentes que usa su template
<code>templateUrl</code>	Ruta al archivo HTML del template
<code>styleUrl</code>	Ruta al archivo CSS (singular, no <code>styleUrls</code>)

Descripción

`selector: 'app-card'`
→ `<app-card></app-card>`

`templateUrl: './card.html'`
← Archivo externo o `template` inline

`styleUrl: './card.css'`
← Estilos encapsulados (ViewEncapsulation)

Ciclo de vida de un componente

Angular llama a métodos del ciclo de vida en orden específico. Los más importantes:

Hook	Cuándo se ejecuta	Uso típico
<code>ngOnChanges()</code>	Cuando cambia una propiedad <code>@Input()</code>	Reaccionar a cambios de entrada
<code>ngOnInit()</code>	Una vez, después del primer <code>ngOnChanges</code>	Inicializar datos, llamadas HTTP
<code>ngAfterViewInit()</code>	Después de que la vista del componente se inicializa	Acceder a elementos del DOM (<code>@ViewChild</code>)
<code>ngOnDestroy()</code>	Justo antes de que Angular destruya el componente	Limpiar suscripciones, timers, evitar memory leaks

Flujo de creación:

```
constructor() → ngOnChanges() → ngOnInit() → ngAfterContentInit() → ngAfterViewInit()
```

Flujo de destrucción:

```
ngOnDestroy()
```

Templates — Binding

Angular ofrece cuatro formas de binding para conectar la clase con el DOM:

Interpolación

```
<h1>{{ title }}</h1>
<p>2 + 2 = {{ 2 + 2 }}</p>
<p>{{ user.name | uppercase }}</p>
```

Del componente → template. Acepta expresiones y pipes.

Property binding

```
<img [src]="imageUrl" [alt]="altText">
<button [disabled]="isLoading">Enviar</button>
<app-card [user]="currentUser"></app-card>
```

Del componente → propiedad HTML o @Input() .

Event binding

```
<button (click)="handleClick($event)">OK</button>
<input (input)="onInput($event)">
<app-card (select)="onCardSelected($event)"></app-card>
```

Del template → componente. Usa \$event .

Two-way binding

```
<input [(ngModel)]="name">
<input [ngModel]="name" (ngModelChange)="name=$event">
```

Combinación de property + event binding (banana-in-a-box).

Templates — Control de flujo

Desde Angular 17, se usan bloques `@` en lugar de directivas estructurales (`*ngIf` , `*ngFor`).

@if / @else

```
@if (isLoggedIn()) {  
  <app-profile />  
} @else {  
  <button (click)="login()">  
    Iniciar sesión  
  </button>  
}
```

@for (con track)

```
@for (item of items(); track item.id) {  
  <div>{{ item.name }}</div>  
} @empty {  
  <p>No hay elementos</p>  
}
```

`track` es obligatorio. Mejora el rendimiento del renderizado.

@switch

```
@switch (status()) {  
  @case ('loading') { <p>Cargando...</p> }  
  @case ('error') { <p>Error</p> }  
  @default { <p>Listo</p> }  
}
```

Variables internas de @for

```
@for (item of items; track item.id; let i = $index, f = $first, l = $last, e = $even, o = $odd) { ... }
```

Comunicación entre componentes

Los componentes se comunican de varias formas según la relación entre ellos:

@Input() — Padre → Hijo

```
// Padre
<app-child [data]="parentData" />

// Hijo
@Input() data: string = '';
// Se puede usar setter para transformar
@Input() set data(v: string) {
  this.processed = v.toUpperCase();
}
```

@Output() — Hijo → Padre

```
// Hijo
@Output() changed = new EventEmitter<string>();
emitChange() { this.changed.emit('nuevo valor'); }

// Padre
<app-child (changed)="onChildChanged($event)" />
```

Servicio compartido

Un servicio inyectable con `providedIn: 'root'` es singleton. Cualquier componente puede leer/escribir estado compartido.

```
@Injectable({ providedIn: 'root' })
export class CartService {
  items = signal<Product[]>([]);
  add(p: Product) { this.items.update(v => [...v, p]); }
}
```

@ViewChild() / @ContentChild()

Acceso directo a un componente hijo desde el padre.

```
@ViewChild(ChildComponent) child!: ChildComponent;
ngAfterViewInit() {
  this.child.someMethod();
}
```


Servicios e Inyección de Dependencias (DI)

Angular tiene un sistema de DI integrado. Los servicios se declaran con `@Injectable` y se inyectan donde se necesitan.

@Injectable

```
@Injectable({
  providedIn: 'root' // singleton global
})
export class ProductService {
  constructor(private http: HttpClient) {}
  list() {
    return this.http.get<Product[]>('/products');
  }
}
```

`providedIn: 'root'` → una sola instancia en toda la aplicación.

`providedIn: 'platform'` → compartida entre múltiples apps.

Inyección

```
// Forma tradicional (constructor)
@Component({...})
export class ProductsPage {
  constructor(private svc: ProductService) {}
}

// Forma moderna (inject)
@Component({...})
export class ProductsPage {
  svc = inject(ProductService);
}
```

`inject()` se recomienda desde Angular 14+. No necesita constructor.

Jerarquía de inyectores: cada componente puede tener su propio inyector hijo, permitiendo alcances diferentes (component-level vs root-level).

Pipes

Los pipes transforman datos en el template. Angular incluye pipes integrados y se pueden crear personalizados.

Pipes integrados

```
{{ today | date:'dd/MM/yyyy' }}  
{{ price | currency:'ARS' }}  
{{ text | uppercase }}  
{{ obj | json }}  
{{ 0.5 | percent }}
```

Pipe personalizado

```
@Pipe({ name: 'truncate', standalone: true })  
export class TruncatePipe  
  implements PipeTransform {  
  transform(v: string, max: number): string {  
    return v.length > max  
      ? v.slice(0, max) + '...'  
      : v;  
  }  
}
```

Uso en template

```
<p>  
  {{ description | truncate:100 }}  
</p>  
  
// Con parámetros  
{{ value | pipe:arg1:arg2 }}
```

Pipe puro vs impuro

Puro (default): solo se ejecuta cuando cambia el valor o los parámetros. Eficiente.

Impuro (`pure: false`): se ejecuta en cada ciclo de detección de cambios. Usar con precaución.

Routing

Angular Router permite navegar entre vistas sin recargar la página. Soporta rutas lazy, guards, parámetros y resolución de datos.

Configuración de rutas

```
export const routes: Routes = [
  { path: '', component: HomePage },
  { path: 'login',
    loadComponent: () =>
      import('./login').then(m => m.LoginPage) },
  { path: 'products',
    loadComponent: () =>
      import('./products').then(m => m.ProductsPage),
    canActivate: [AuthGuard] },
  { path: '**', redirectTo: '' },
];
```

Router outlet y navegación

```
<!-- template principal -->
<router-outlet />

<!-- navegación declarativa -->
<a routerLink="/products">Productos</a>

<!-- navegación imperativa -->
export class LoginPage {
  router = inject(Router);
  goHome() {
    this.router.navigate(['/']);
  }
}
```

Parámetros de ruta

```
// Ruta: products/:id
// Navegar:
this.router.navigate(['products', id]);
// Leer:
id = input.required<string>();
// o
route = inject(ActivatedRoute);
route.paramMap.subscribe(...)
```

Query params

```
// products?page=1
this.router.navigate([], {
  queryParams: { page: 1 }
});
// Leer:
route = inject(ActivatedRoute);
route.queryParamMap.subscribe(...)
```

Lazy loading

Con `loadComponent` el componente se carga solo cuando se navega a esa ruta. Reduce el bundle.

```
{
  path: 'admin',
  loadComponent: () =>
    import('./admin').then(m => m.AdminPage)
}
```

Guards

Los guards controlan el acceso a las rutas. Son funciones que devuelven `true` (permite acceso), `false` (deniega) o una `UrlTree` (redirige).

CanActivate

```
export const authGuard: CanActivateFn =
() => {
  const token = localStorage
    .getItem('access_token');
  if (!token) {
    return inject(Router)
      .parseUrl('/login');
  }
  return true;
};
```

CanActivateChild

```
export const adminGuard: CanActivateFn =
() => {
  const user = JSON.parse(
    localStorage.getItem('user')!
  );
  return user?.role === 'admin'
    ? true
    : inject(Router).parseUrl('/');
};
```

Otros guards

- `CanDeactivate` — confirmar salida (formularios sucios)
- `CanMatch` — evitar que una ruta coincida
- `CanLoad` — evitar lazy loading
- `Resolve` — precargar datos antes de activar

Se aplican en el array `canActivate` de la ruta:

```
{ path: 'admin', canActivate: [authGuard, adminGuard], loadComponent: ... }
```

HttpClient e Interceptors

HttpClient es el servicio oficial para hacer peticiones HTTP. Los interceptors permiten modificar requests/responses globalmente.

HttpClient

```
import { provideHttpClient }
  from '@angular/common/http';

// app.config.ts
providers: [
  provideHttpClient(
    withInterceptors([jwtInterceptor])
  )
];

// Uso en servicio
@Injectable({providedIn:'root'})
export class ApiService {
  http = inject(HttpClient);
  getProducts() {
    return this.http.get('/products');
  }
}
```

Los métodos devuelven `Observable` (RxJS).

Interceptor funcional

```
export const jwtInterceptor:
  HttpInterceptorFn = (req, next) => {
  const token = localStorage
    .getItem('access_token');
  if (token) {
    req = req.clone({
      headers: req.headers
        .set('Authorization',
          `Bearer ${token}`)
    });
  }
  return next(req);
};
```

Los interceptores funcionales reemplazan a los class-based desde Angular 15. Se ejecutan en orden.

GET → `http.get(url, options)`

POST → `http.post(url, body)`

PUT/PATCH/DELETE →
`http.put/patch/delete(...)`

Señales (Signals)

API de reactividad introducida en Angular 16, estable desde Angular 17. Alternativa más simple que RxJS para estado síncrono.

signal()

```
import { signal, computed } from '@angular/core';

contador = signal(0);
nombre = signal('Juan');
items = signal<string[]>([]);

// Leer
console.log(this.contador());

// Escribir
this.contador.set(5);
this.contador.update(v => v + 1);
this.items.update(v => [...v, 'nuevo']);
```

computed()

```
precio = signal(100);
iva = computed(() =>
  this.precio() * 0.21
);
total = computed(() =>
  this.precio() + this.iva()
);

// Solo lectura. Se recalcula
// automáticamente cuando
// cambian sus dependencias.
```

effect()

```
effect(() => {
  console.log(
    'El contador cambió a:',
    this.contador()
  );
});

// Se ejecuta cuando cambia
// cualquier signal leída
// dentro del effect.
// Usar con cuidado: no en
// flujos de cambio de estado.
```

Detección de cambios con signals

Cuando una signal cambia, Angular solo revisa los componentes que la usan. Más eficiente que Zone.js tradicional. Se puede combinar con `ChangeDetectionStrategy.OnPush`.

Standalone vs NgModules

Angular tradicional usaba NgModules para organizar el código. Desde Angular 14+ los componentes pueden ser standalone. Angular 17+ recomienda standalone como default.

Standalone (actual)

```
@Component({
  standalone: true,
  selector: 'app-user',
  imports: [
    CommonModule,
    UserService
  ],
  templateUrl: './user.html'
})
export class UserComponent {
  svc = inject(UserService);
}
```

Ventajas:

- Sin NgModules, menos boilerplate
- Cada componente declara sus imports
- Lazy loading más simple
- Mejor tree-shaking

NgModule (legacy)

```
@NgModule({
  declarations: [UserComponent],
  imports: [CommonModule],
  providers: [UserService],
  exports: [UserComponent]
})
export class UserModule {}
```

Desventajas:

- Más archivos y configuración
- Acoplamiento entre módulos
- Más difícil de tree-shake
- Curva de aprendizaje más alta

El proyecto frontend de api-c usa exclusivamente standalone components.

Aplicación al proyecto

Cada concepto de Angular se refleja directamente en el código de `front/src/app/` :

Concepto Angular	Implementación en el proyecto
Componentes standalone	Todas las páginas (login, register, profile, products, categories, admin-users) y el navbar son standalone.
Template con <code>@if</code> / <code>@for</code>	Listas de productos, categorías y usuarios usan <code>@for</code> ; estados de carga/error con <code>@if</code> .
Routing lazy	<code>app.routes.ts</code> usa <code>loadComponent</code> para cargar páginas bajo demanda.
Guards	<code>auth.guard.ts</code> protege rutas autenticadas; <code>admin.guard.ts</code> restringe acceso a admins.
HttpClient + Interceptor	<code>jwt.interceptor.ts</code> agrega el token JWT a cada request automáticamente.
Servicios	<code>auth.service.ts</code> , <code>products.service.ts</code> , <code>categories.service.ts</code> , <code>users.service.ts</code> encapsulan llamadas HTTP.
Signals	Estado reactivo local en cada página (ej: <code>products = signal<Product[]>([])</code>).
<code>inject()</code>	Todos los componentes y servicios usan <code>inject()</code> en vez de constructor DI.
Bootstrap 5	CSS importado en <code>angular.json</code> . Clases Bootstrap en templates para layout responsivo.